

Karma Jasmine Tests

<https://www.htmlgoodies.com/html5/javascript/spy-on-javascript-methods-using-the-jasmine-testing-framework.html>

In Jasmine, mocks are referred to as spies. There are two ways to create a spy in Jasmine: `spyOn()` can only be used when the method already exists on the object, whereas `jasmine.createSpy()` will return a brand new function:

```
?
1 //spyOn(object, methodName) where object.method() is a
2 function
3 spyOn(obj, 'myMethod')
4
5 //jasmine.createSpy(stubName);
6 var myMockMethod = jasmine.createSpy('My Method');
```

As we'll soon see, both of the above methods have their place in your unit tests.

Using the `spyOn()` Method

As mentioned above, `spyOn()` can only be used when the method already exists on the object. For simple tests, this is your best bet.

Our test cases all feature the following `Person` object. It has a couple of attributes, a getter and setter for the name, and two public methods:

```
?
1
2 var Person = function() {
3     //defaults
4     var _age = 0,
5         _name = 'John Doe';
6
7     this.initialize = function(name, age) {
8         _name = name || _name;
9         _age = age || _age;
10    };
11    if (arguments.length) this.initialize();
12
13    //getters and setters
14    this.getName = function() { return _name; };
15    this.setName = function(name) { _name = name; };
16
17    //public methods
18    this.addBirthDay = function() { _age++; };
19    this.toString = function() { return 'My name is ' + this.getName() + ' and ' +
20    ' years old.'; };
21    };
22 }
```

Say that we want to verify that the `toString()` method was calling `getName()`. We would instantiate the `Person` as usual, but before calling `toString()`, we would call `spyOn()`, passing in the person instance and the name of the method that we want to spy on ('`getName`'). We can then call jasmine matchers to see what happened. The simplest test is to check that `getName()` was in fact called:

```
?
1 describe("Person toString() Test", function() {
2     it("calls the getName() function", function() {
3         var testPerson = new Person();
4         spyOn(testPerson, "getName");
5         testPerson.toString();
6         expect(testPerson.getName).toHaveBeenCalled();
7     });
8 }
```

```
7 });  
8
```

But that's just the beginning. We can run other tests on our spied function, such as what arguments it was called with. The `toHaveBeenCalled()` method accepts a value to be compared against the method's `arguments` attribute. Conversely, we can test that the function was called without any parameters by calling `toHaveBeenCalled()` without a value:

```
?  
1 describe("Person toString() Test", function() {  
2     var testPerson;  
3     beforeEach(function() { testPerson = new Person(); });  
4     afterEach(function() { testPerson = undefined; });  
5  
6     it("calls the getName() function", function() {  
7         spyOn(testPerson, "getName");  
8         testPerson.toString();  
9         expect(testPerson.getName).toHaveBeenCalled();  
10    });  
11  
12    it("Method getName() was called with zero arguments",  
13    function() {  
14        // Ensure the spy was called with the correct number of  
15        arguments  
16        // In this case, no arguments  
17        expect(testPerson.getName).toHaveBeenCalled();  
18        // this also works  
19        //  
20        expect(testPerson.getName.mostRecentCall.args.length).toEqual(0);  
21    });  
22 });
```

Creating Our Own Spy Method

Sometimes, it may be beneficial to completely replace the original method with a fake one for testing. Perhaps the original method takes a long time to execute, or it depends on other objects that aren't available in the test context. Jasmine lets us handle this issue by creating a fake method using `jasmine.createSpy()`. Here's how to substitute a fake `getName()` for the real one:

```
?  
1 describe("Person toString() Test with Fake getName() Method",  
2 function() {  
3     it("calls the fake getName() function", function() {  
4         var testPerson = new Person();  
5         testPerson.getName = jasmine.createSpy("getName spy");  
6         testPerson.toString();  
7         expect(testPerson.getName).toHaveBeenCalled();  
8     });  
9 });
```

Unlike `spyOn()`, creating a fake method circumvents the original method so that it is not called during tests. Thus, the alert in `getName()` below will not appear:

```
?  
1 var Person = function() {  
2     //...  
3     this.getName = function() {  
4         alert("You called?"); //won't be called  
5         return _name;  
6     };  
7 }
```

```

5         };
6         //...
7     };
8     describe("Person toString() Test with Fake getName() Method",
9     function() {
10         it("calls the fake getName() function", function() {
11             var testPerson = new Person();
12             testPerson.getName = jasmine.createSpy("getName() spy");
13             testPerson.toString();
14             expect(testPerson.getName).toHaveBeenCalled();
15         });
16     });
17

```

Modifying the Fake Method

If your method is being called by another method, you may want it to return something. You can tell Jasmine what to return using the `andReturn(value)` method:

```

?
1 testPerson.getName = jasmine.createSpy("getName()
  spy").andReturn("Bobby");

```

And finally, here's a way to substitute an entirely different method body for the original:

```

?
1 // defining a spy on an existing property: testPerson.getName() calls an
  anonymous function
2 testPerson.getName = jasmine.createSpy("getName() spy").andCallFake(function()
3 {
4     console.log("Hello from getName()");
5     return "Bobby";
6 });

```

The above function not only returns "Bobby" each time, but it also logs a message to the console. That would be a little harder to do with the original function.